

Master Guide — Complete Project Reference

The single document to learn this entire project: the exact order to read every file, high- and low-level design, all diagrams, every script, all code, and the full record of what was built and fixed.

PART A — The Learning Sequence (read files in THIS order)

Follow these 10 stages top to bottom. Each builds on the previous.

Stage 0 — See it run first

#	File / Action	Why
0.1	<code>docker compose up -d</code>	Watch the whole system boot before reading any code
0.2	<code>curl localhost:8000/api/products</code>	Prove the gateway → service path works

Stage 1 — The map

#	File	What you learn
1.1	<code>README.md</code>	Project overview, service list, quick start
1.2	<code>docs/MASTER_GUIDE.md</code> (this file)	The whole picture
1.3	<code>docs/ARCHITECTURE.md</code>	System diagrams + data flow

Stage 2 — How traffic enters

#	File	What you learn
2.1	<code>docker-compose.yml</code>	Every container, port, env var, dependency
2.2	<code>kong/kong.yml</code>	Gateway routing: which URL → which service (<code>strip_path:false</code>)

Stage 3 — One service, end to end (use `user-service` — simplest)

#	File	Layer
3.1	<code>user-service/src/main/resources/application.yml</code>	Config: DB, Redis, JWT
3.2	<code>.../model/User.java</code>	Entity (maps to <code>users</code> table)
3.3	<code>.../repository/UserRepository.java</code>	DB queries (Spring Data JPA)
3.4	<code>.../service/UserService.java</code>	Business logic

3.5	<code>.../controller/UserController.java</code>	REST endpoints
3.6	<code>.../config/SecurityConfig.java</code> + <code>JwtAuthenticationFilter.java</code>	JWT auth
3.7	<code>.../resources/db/migration/V1__init.sql</code>	Flyway schema

Stage 4 — The event backbone (`order` → `inventory` → `payment` → `notification`)

#	File	What you learn
4.1	<code>order-service/.../kafka/OrderEventPublisher.java</code>	Publishing <code>order.created</code>
4.2	<code>inventory-service/.../kafka/InventoryEventConsumer.java</code>	Reserving stock
4.3	<code>inventory-service/.../service/InventoryService.java</code>	Redis lock + atomic upsert
4.4	<code>payment-service/.../kafka/PaymentEventConsumer.java</code>	Charging only after stock OK
4.5	<code>order-service/.../kafka/OrderEventConsumer.java</code>	Completing saga → PAID / CANCELLED
4.6	<code>notification-service/.../kafka/NotificationConsumer.java</code>	Emails via SES

Stage 5 — Caching & concurrency

#	File	What you learn
5.1	<code>product-service/.../service/ProductService.java</code>	Redis cache (15-min TTL)
5.2	<code>inventory-service/.../service/InventoryService.java</code>	Redis distributed lock (oversell prevention)

Stage 6 — Containerisation

#	File	What you learn
6.1	<code>*/Dockerfile</code>	How each service becomes an image
6.2	<code>pom.xml</code> (root)	Multi-module reactor, Flyway, JaCoCo

Stage 7 — Kubernetes

#	File	What you learn
---	------	----------------

7.1	k8s/namespace/namespace.yaml	Namespaces
7.2	k8s/postgres/postgres.yaml	StatefulSets (5 DBs)
7.3	k8s/kafka/kafka.yaml + redis/redis.yaml	Stateful infra
7.4	k8s/kong/kong.yaml	Gateway + LoadBalancer service
7.5	k8s/services/deployments.yaml	Deployments + HPA
7.6	k8s/services/configmap.yaml + secrets.example.yaml	Config vs secrets
7.7	k8s/monitoring/monitoring.yaml	Prometheus, Grafana, Loki

Stage 8 — AWS via Floci (scripts)

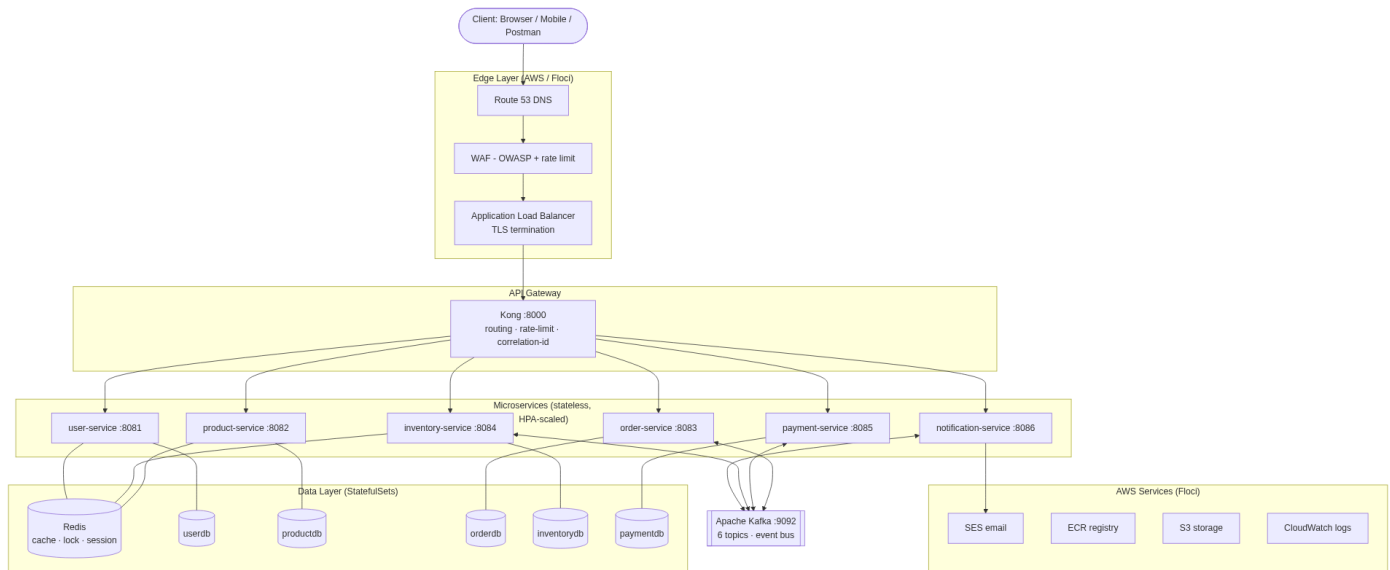
#	File	What you learn
8.1	scripts/01-vpc.sh	VPC, subnets, SG, NAT
8.2	scripts/02-alb.sh	ALB, WAF, ACM, Route53
8.3	scripts/03-ecr.sh	Build + push images to ECR
8.4	scripts/04-eks.sh	EKS cluster + node groups
8.5	scripts/05-deploy.sh	Deploy manifests + register in ALB
8.6	scripts/06-teardown.sh	Delete everything

Stage 9 — CI/CD & governance

#	File	What you learn
9.1	.github/workflows/ci-cd.yml	Full pipeline
9.2	.github/smoke-test.sh	16-endpoint E2E saga test
9.3	.github/workflows/codeql.yml	Security scanning
9.4	.github/dependabot.yml	Dependency automation
9.5	scripts/setup-github-governance.sh	Environments + branch protection
9.6	docs/CI_CD.md , DEPLOYMENT.md , AUTOSCALING.md , MONITORING.md	Deep dives

PART B — High-Level Design (HLD)

HLD = the big picture: major components and how they talk. No class detail.



HLD principles

Principle	How it's applied
Database per service	5 separate Postgres DBs; no shared tables
API Gateway	Kong is the single entry; services never exposed directly
Event-driven	Services communicate via Kafka events, not direct calls (loose coupling)
Stateless services	Any pod can serve any request; state lives in DB/Redis/Kafka
CQRS-lite / Saga	Order flow is a choreographed saga across services

PART C — Low-Level Design (LLD)

LLD = the internals: layers, classes, tables, and exact message flow.

C1. Standard service layering (every service follows this)

 diagram

Layer	Annotation	Responsibility	Must NOT
Controller	<code>@RestController</code>	HTTP mapping, status codes	contain business logic
Service	<code>@Service</code> <code>@Transactional</code>	business rules, orchestration	know about HTTP
Repository	<code>extends JpaRepository</code>	DB access only	contain logic
Model	<code>@Entity</code>	table mapping	contain behaviour
DTO	plain + <code>@Valid</code>	request/response shape	expose entities directly
Kafka	<code>@KafkaListener</code> / publisher	async events	block on responses

C2. Database schemas (LLD data model)

```
userdb.users          (id, name, email UNIQUE, password_hash, role, active, created_at)
productdb.products    (id, name, price, category, description, created_at)
orderdb.orders         (id, user_id, status, total_amount, shipping_address, created_at, updated_at)
orderdb.order_items    (id, order_id FK, product_id, product_name, quantity, price)
inventorydb.inventory (id, product_id UNIQUE, quantity, reserved, version) ← @Version optimistic lock
paymentdb.payments     (id, order_id, user_id, amount, status, transaction_id, created_at)
```

C3. The Order Saga (detailed LLD sequence)

 diagram

Order status machine: `PENDING → PAID` (happy) or `PENDING → CANCELLED` (no stock / payment fail).

C4. Kafka topics (LLD messaging)

Topic	Publisher	Consumers	Payload keys
order.created	order	inventory, payment*, notification	orderId, userId, amount, items
inventory.updated	inventory	order, payment	orderId, success, reason
payment.processed	payment	order, notification	orderId, success, transactionId
order.cancelled	order	inventory, notification	orderId, reason
product.created	product	inventory	productId

*payment only acts on `inventory.updated{success:true}` , never on raw `order.created` .

C5. Redis usage (LLD)

Key pattern	Service	Purpose	TTL
session:{email}	user	JWT session	24h
product:{id}	product	read cache	15 min
inventory:lock:{productId}	inventory	distributed lock (oversell guard)	30s

PART D — Every Script Explained

Script	Does	Key AWS concepts
<code>scripts/01-vpc.sh</code>	VPC 10.0.0.0/16, 2 public + 2 private subnets, IGW, NAT, route tables, 3 security groups	CIDR, public/private subnet, NAT, SG
<code>scripts/02-alb.sh</code>	ALB in public subnets, target group (IP mode), listeners 80→443, WAF, ACM cert, Route53	ALB, listeners, target groups, WAF, ACM
<code>scripts/03-ecr.sh</code>	Create 6 ECR repos, <code>mvn package</code> , <code>docker build</code> , push to Floci ECR (localhost:5100)	ECR, docker login, image tags
<code>scripts/04-eks.sh</code>	IAM roles, EKS cluster, 2 node groups, kubeconfig, auto-writes k3s registries.yaml for ECR pulls	EKS, IAM roles, node groups
<code>scripts/05-deploy.sh</code>	Apply all k8s manifests in order, register Kong pod IPs in ALB target group	kubectl apply order, target registration
<code>scripts/06-teardown.sh</code>	Delete EKS, ALB, WAF, VPC, ECR	clean teardown
<code>scripts/setup-github-governance.sh</code>	Create <code>production + dev</code> environments, branch protection on main/develop via GitHub API	environments, required reviewers, branch rules
<code>.github/smoke-test.sh</code>	Drive all 16 REST endpoints through Kong + verify order→PAID saga; runs local (KONG_URL) or CI (port-forward)	E2E testing
<code>scripts/generate-pdf-docs.js</code>	Combine all docs into one printable HTML	docs tooling

PART E — What We Built & Fixed (the full journey)

Phase 1 — Get endpoints working on Docker

Bug found	Fix
Kafka advertised <code>localhost:9092</code> in compose	changed to <code>kafka:9092</code>
Product Redis cache crash (serialization)	<code>Product</code> implements <code>Serializable</code>
Order JSON infinite recursion	<code>@JsonIgnore</code> on <code>OrderItem.order</code>
No JWT filter in user-service	added <code>JwtAuthenticationFilter</code> + stateless <code>SecurityConfig</code>
500s instead of proper codes	added <code>GlobalExceptionHandler</code> (404/401/409)
Kong 404 on all routes	added <code>strip_path: false</code> to all 6 routes
Payment charged even when out of stock	rewired: payment only on <code>inventory.updated{success:true}</code>

Phase 2 — Kubernetes

Bug	Fix
Kafka crash-loop (missing <code>KAFKA_NODE_ID</code>)	merged duplicate <code>env:</code> block
Kafka <code>\$(POD_NAME)</code> unresolved	reordered env vars
Promtail crash (missing ConfigMap)	added <code>promtail-config</code> + RBAC
Secrets in git	removed; inject via <code>kubectl create secret</code>

Phase 3 — CI/CD

Bug	Fix
<code>configure-aws-credentials</code> rejects test creds	export env vars directly for Floci
Kind too slow for CI	switched to k3d (fast k3s)
k3d-action pinned v5.4.6 URL 404	install latest k3d directly
CI smoke test 503	wait for ALL services before testing

Trivy re-downloads DB	cached official action
-----------------------	------------------------

Phase 4 — Completing the app (found via full E2E test)

Gap	Fix
No way to add stock (every order cancelled)	added <code>POST /api/inventory/{id}/restock</code>
Paid orders stuck at PENDING forever	added <code>payment.processed</code> listener → mark PAID
<code>restock</code> 500 under concurrency (duplicate key race)	atomic <code>INSERT ... ON CONFLICT upsert</code>

Phase 5 — Governance

- `production` environment with required reviewer (approval gate)
 - Branch protection: PR + 6 test checks + no force-push on main & develop
 - Repo made public to unlock free branch protection
 - **Verified end-to-end:** develop auto-deploy green, main gated deploy approved & green
-

PART F — How To Run / Verify Everything

```
# 1. Local (fastest)
docker compose up -d
KONG_URL=http://localhost:8000 bash .github/smoke-test.sh    # expect 17/17

# 2. Full AWS simulation (Floci)
bash scripts/01-vpc.sh && bash scripts/02-alb.sh && bash scripts/03-ecr.sh
bash scripts/04-eks.sh && bash scripts/05-deploy.sh
kubectl get pods -n ecommerce

# 3. CI/CD
git push origin develop      # → tests + k8s deploy + smoke test
# open PR develop→main → merge → approve production gate

# 4. Monitoring
kubectl port-forward svc/grafana 3000:3000 -n monitoring
```

Definition of done (all achieved): every endpoint tested on every deploy, full saga reaches PAID, real Kubernetes deploy green on both environments, production gated behind human approval.